

15주차

1. BERT란?

BERT (Bidirectional Encoder Representations from Transformers) 는 2018년 Google이 발표한 **Transformer 기반 사전학습 언어 모델**이다.

- Transformer의 **인코더(Encoder)** 구조만 사용
- 문장을 **양방향(Bidirectional)** 으로 이해
- 대규모 말뭉치로 사전 학습 → 다운스트림 태스크에 미세 조정(Fine-tuning)

2. BERT의 핵심 특징: 양방향 인코딩

기존 모델의 한계

- RNN, LSTM, GPT 계열 → **한 방향**
- 단어 의미를 파악할 때 **이전 단어만** 참고

BERT의 차별점

- 문장을 좌 → 우, 우 → 좌 동시에 학습
- 특정 단어를 이해할 때 **앞·뒤 문맥을 모두 활용**

예시

I'm learning PyTorch → **learning** 은 *I'm*과 *PyTorch* 둘 다 참고해서 의미를 파악
문맥 이해 능력이 크게 향상됨

3. BERT의 구조

✓ Transformer Encoder Only

- 디코더
- 인코더 블록 **12개 (BERT-base 기준)**

✓ 인코더 내부 구성

각 Encoder Block은:

1. **Multi-Head Self-Attention**
2. **Feed Forward Network**
3. Add & LayerNorm
4. Dropout

➡ 문장 내 모든 단어 간 관계를 동시에 학습

4. 입력 임베딩 구조

BERT의 입력 임베딩은 3가지의 합으로 구성됨

① Token Embedding

- 각 단어(토큰)의 의미

② Position Embedding

- 단어의 위치 정보 (순서 정보)

③ Token Type Embedding

- 문장 A / 문장 B 구분 (NSP용)

➡ 세 임베딩을 더한 후 LayerNorm + Dropout 적용

5. 특수 토큰 (Special Tokens)

토큰	역할
[CLS]	문장 전체 대표 벡터 (분류 작업에 사용)
[SEP]	문장 구분
[MASK]	MLM 학습용 마스킹 토큰

입력 예시

[CLS] 문장1 [SEP] 문장2 [SEP]

6. BERT 사전 학습 방법 (Pre-training)

① MLM (Masked Language Modeling)

- 전체 토큰 중 **15%**를 마스킹
- 그중:
 - 80% → [MASK]
 - 10% → 랜덤 단어
 - 10% → 원래 단어 유지

목적

➡ 양방향 문맥을 이용해 가려진 단어 예측

② NSP (Next Sentence Prediction)

- 두 문장이 주어졌을 때:
 - 두 번째 문장이 첫 번째 다음 문장인지 판단

목적

➡ 문장 간 관계 이해 (QA, NLI에 중요)

7. BERT의 출력 벡터 활용

- 문장 분류 → [CLS] 토큰 벡터 사용
 - 토큰 단위 예측 (NER 등) → 모든 토큰 벡터 사용
-

8. Fine-tuning 방식

- 사전학습된 BERT 위에 **Task-specific Layer** 추가
- 전체 모델을 함께 미세 조정

예시

- 감성 분석
 - 문장 분류
 - 질의응답
 - 개체명 인식
-

9. 실습 코드 흐름 정리

(1) 데이터 로드

- Naver 영화 리뷰 (Korpora)
- 20,000개 샘플 → **6:2:2 분할**

(2) 토큰나이징

```
BertTokenizer.from_pretrained("bert-base-multilingual-cased")
```

- 대소문자 유지
- 다국어 지원 (104개 언어)

출력:

- `input_ids`
- `attention_mask`
- `labels`

(3) Dataset & DataLoader

- `RandomSampler` → 학습
- `SequentialSampler` → 검증/테스트

(4) 모델 선언

```
BertForSequenceClassification
```

구성:

- BERT Encoder
- Pooler (Tanh)
- Dropout
- Classifier

(5) Optimizer

- **AdamW**

- Adam + Weight Decay
- 대규모 모델 안정적 학습

10. 학습 결과 해석

- Epoch 증가 → **Train / Val Loss** 감소
- Val Accuracy 증가 → 과적합
- Test Accuracy ≈ **80%**

➡ 모델이 안정적으로 학습되었음을 의미

BART 개념 정리 (핵심 요약)

1. BART란?

BART (Bidirectional Auto-Regressive Transformer) 는

2019년 **Meta(Facebook) FAIR** 연구소에서 발표한 **Transformer** 기반 사전학습 언어 모델이다.

- BERT의 인코더 + GPT의 디코더를 결합
- **Sequence-to-Sequence (Seq2Seq)** 구조
- 노이즈 제거 오토인코더(**Denoising Autoencoder**) 방식으로 사전 학습

➡ 이해(BERT) + 생성(GPT) 을 모두 잘하는 모델

2. BART의 핵심 아이디어

💡 “망가진 문장을 원래 문장으로 복원하라”

1. 입력 문장에 **의도적으로 노이즈** 추가
2. 노이즈가 섞인 문장을 **인코더에 입력**
3. 디코더가 **원래 문장을 생성**하도록 학습

➡ 문장 의미 + 구조 + 생성 능력 모두 학습

3. BART와 BERT, GPT 비교

모델	구조	특징
BERT	Encoder only	문장 이해에 강함 (분류, 분석)
GPT	Decoder only	문장 생성에 강함
BART	Encoder + Decoder	이해 + 생성 모두 가능

핵심 차이

- BERT 문장 생성
- BART 문장 생성, 요약, 번역

4. BART 모델 구조

✓ Encoder-Decoder 구조

- 인코더: 입력 문장 이해 (양방향)
- 디코더: 문장 생성 (자기회귀)

✓ 어텐션 구조 특징

- 인코더의 마지막 층 ↔ 디코더 각 층 사이만 cross-attention 수행
- 일반 Transformer보다 메모리 사용량 감소

5. BART 사전 학습 방법 (Pre-training)

✓ Denoising Autoencoder

입력 문장에 다양한 노이즈 기법을 적용한 뒤,
디코더가 원본 문장 전체를 생성하도록 학습

6. BART의 노이즈 기법 정리

① 토큰 마스킹 (Token Masking)

- BERT의 MLM과 유사
- 일부 토큰을 [MASK] 로 치환

효과

→ 중요한 단어를 문맥으로 추론

② 토큰 삭제 (Token Deletion)

- 일부 토큰을 완전히 제거
- 모델이 어디가 빠졌는지까지 추론

효과

→ 불필요한 정보 필터링 + 일반화 성능 향상

③ 문장 순열 (Sentence Permutation)

- 문장 순서를 섞음
- 모델이 원래 문장 순서 복원

효과

→ 문장 간 논리적 흐름 학습

④ 문서 회전 (Document Rotation)

- 문서 시작 위치를 임의로 변경
- 모델이 문서 시작점 인식

효과

→ 문서 구조 이해 능력 향상

⑤ 텍스트 채우기 (Text Infilling)

- 연속된 토큰 구간(span)을 하나의 [MASK]로 치환
- 구간 길이는 **Poisson 분포($\lambda=3$)** 사용

효과

→ 단어 단위가 아닌 구문·구간 단위 이해

7. BART 미세 조정(Fine-tuning) 방식

BART는 **Encoder + Decoder** 구조이므로

다운스트림 태스크마다 입력/출력 형태가 다름

✓ 문장 분류

- 입력 문장을 인코더와 디코더에 동일하게 입력

- 디코더 마지막 hidden state → 분류기

BERT의 [CLS] 역할을

디코더 마지막 상태가 수행

✓ 토큰 분류 (NER 등)

- 디코더 각 시점의 hidden state 사용
 - 토큰 단위 예측 가능
-

✓ 문장 생성 / 요약 / 번역

- 인코더: 원문 입력
- 디코더: `<s>` 부터 시작해 문장 생성

BART의 가장 강력한 분야

8. BART가 잘하는 작업

- 문서 요약 (Summarization)
- 추상적 질의응답 (Abstractive QA)
- 기계 번역
- 문장 생성

➡ “입력을 변형해 새로운 텍스트를 만드는 작업”에 최적

9. 실습 흐름 요약 (뉴스 요약 예제)

(1) 데이터셋

- Hugging Face `datasets`
 - `argilla/news-summary`
 - 뉴스 본문 → 요약문
-

(2) 데이터 분할

- 5,000개 샘플
- 6 : 2 : 2 (train / valid / test)

(3) 토큰나이징

```
BartTokenizer.from_pretrained("facebook/bart-base")
```

- 입력(text): 뉴스 본문
- 출력(label): 요약 문장
- 길이가 다르므로 **padding 필요**

(4) 패딩 처리

```
padding_value = -100
```

이유

- CrossEntropyLoss에서 **100은 무시**
- 패딩 토큰이 손실 계산에 영향

(5) 모델 학습

- Encoder: 뉴스 이해
- Decoder: 요약 문장 생성
- Teacher Forcing 사용

10. BART의 장점 정리

- 양방향 문맥 이해 + 문장 생성 가능
- 다양한 노이즈 → 강한 일반화 성능
- 긴 문서 처리 및 구조 이해 우수

1. BART 조건부 생성 모델 선언

BartForConditionalGeneration

```
BartForConditionalGeneration.from_pretrained("facebook/bart-base")
```

- 조건부 생성(Conditional Generation) 모델
- 입력 시퀀스를 조건으로 출력 시퀀스를 생성
- 사용 작업:
 - 문장 요약
 - 기계 번역
 - 질의응답 (QA)

BERT와 차이

- BERT: 분류/이해 중심
- BART: "입력 → 출력" 생성 중심

2. bart-base vs bart-large

모델	인코더/디코더 층
bart-base	6 / 6
bart-large	12 / 12

- 실습에서는 학습 속도와 자원 절약을 위해 `bart-base` 사용

3. Optimizer 설정

```
optimizer = AdamW(model.parameters(), lr=5e-5, eps=1e-8)
```

- **AdamW**: Adam + Weight Decay
- 대규모 Transformer 학습에 안정적
- BERT 실습과 동일한 설정 → 일관성 유지

BART 모델 구조 이해

핵심 구조

```
shared
└ encoder (6 layers)
```

```
└ decoder (6 layers)
  └ lm_head
```

✓ shared embedding

- 인코더와 디코더가 같은 토큰 임베딩 공유
- 의미 공간을 동일하게 유지 → 생성 품질 향상

✓ encoder

- 입력 문장 이해
- 마지막 encoder layer 출력 → decoder의 cross-attention에 사용

✓ decoder

- 이전 출력 토큰 + encoder 정보 활용
- 다음 토큰을 자기회귀적으로 생성

✓ lm_head

- 디코더 출력 → vocabulary 크기로 projection
- 실제 단어 확률 분포 생성

5. 요약 데이터셋 입력/출력 구조

✓ 입력 (encoder input)

- 뉴스 본문 텍스트
- `input_ids`, `attention_mask`

✓ 출력 (decoder target)

- 요약 문장
- 길이가 다르므로 padding 필요

6. 패딩 값이 -100인 이유

```
padding_value = -100
```

- PyTorch `CrossEntropyLoss` 에서
 - 100은 loss 계산에서 무시
- 패딩 토큰이 학습에 영향을 주지 않도록 설계

 요약:

| "실제 요약 토큰만 손실 계산에 사용"

7. ROUGE 평가 지표 개념

ROUGE란?

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

→ 생성된 요약문과 정답 요약문이 얼마나 비슷한지 평가

① ROUGE-1

- Unigram(단어 단위) 기준
- 핵심 단어 포함 여부 평가

② ROUGE-2

- Bigram(연속 2단어) 기준
- 문장 자연스러움 평가

③ ROUGE-L

- LCS (최장 공통 부분 수열) 기반
- 문장 구조 유사성 반영

④ ROUGE-LSUM

- 문장 단위 LCS 결합
 - 요약 작업에서 가장 많이 사용
-

8. ROUGE 계산 흐름 (calc_rouge 함수)

① 모델 출력 처리

```
preds = logits.argmax(axis=-1)
```

- 각 토큰 위치에서 가장 높은 확률 단어 선택

② 패딩 복원

```
labels = np.where(labels != -100, labels, pad_token_id)
```

- 평가를 위해 -100 → pad 토큰으로 복구

③ 텍스트 복원

```
tokenizer.batch_decode(...)
```

- 토큰 ID → 실제 문장

④ ROUGE 계산

```
rouge_score.compute(predictions, references)
```

9. 학습 결과 해석

✓ 학습 로그 예시

```
Epoch 1: Val ROUGE 0.256  
Epoch 2: Val ROUGE 0.261  
Epoch 3: Val ROUGE 0.254
```

- ROUGE 값이 **0~1 사이**
- 1에 가까울수록 정답 요약과 유사
- 작은 데이터셋(3,000개 학습) 고려 시 **중간 수준 성능**

10. Test 성능

```
Test ROUGE-2 Score : 0.2699
```

해석

- 데이터 규모가 작음에도
- 요약 핵심 의미를 어느 정도 포착
- 실제 요약 품질은 정성 평가 병행 필요

11. 파이프라인 기반 요약 비교

```
pipeline("summarization")
```

- 실제 뉴스 본문 입력
- 모델 요약 vs 정답 요약 비교

관찰 포인트

- 핵심 주제 유지 여부
- 고유명사 보존
- 불필요한 정보 제거 여부

 ROUGE 점수 + 사람 눈으로 직접 비교가 중요

12. BART 요약 실습의 핵심 정리

왜 BART가 요약에 강한가?

- Encoder-Decoder 구조
- 다양한 노이즈 기반 사전 학습
- 전체 문맥 이해 + 문장 생성 가능

평가에서 중요한 점

- ROUGE는 완벽한 지표
- 반드시 정성 평가와 함께 사용

1. ELECTRA란?

ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately)

- 2020년 **Google**이 발표한 **Transformer 기반 사전학습 모델**
- 목표: **BERT보다 더 효율적인 사전 학습**

 핵심 차별점

| [MASK]를 맞히는 대신, “토큰이 바뀌었는지”를 맞힌다

2. BERT vs ELECTRA 핵심 차이

구분	BERT	ELECTRA
사전학습	MLM (마스크 예측)	RTD (토큰 교체 판별)
학습 신호	일부 토큰만	모든 토큰
효율성	상대적으로 낮음	매우 높음
구조	Encoder	Encoder (최종 사용)

 같은 데이터·비슷한 파라미터 수에서 ELECTRA가 더 높은 성능

3. ELECTRA의 핵심 아이디어

Generator + Discriminator 구조

- **Generator (생성자)**
 - BERT와 동일한 **MLM**
 - 일부 토큰을 마스크 → 원래 토큰 예측
- **Discriminator (판별자)**
 - 각 토큰이
원본 토큰인지
Generator가 바꾼 토큰인지
를 판별

 이 방식 → **RTD (Replaced Token Detection)**

4. RTD (Replaced Token Detection)

✓ 학습 과정

1. 입력 문장의 **15% 토큰 마스킹**
2. Generator가 마스크 토큰 예측
3. 예측 결과로 문장을 복원
4. Discriminator가 **각 토큰마다**
 - 원본(Real)
 - 교체(Fake)여부를 분류

➡ 모든 토큰이 학습에 기여

➡ BERT보다 훨씬 효율적인 학습

5. ELECTRA vs GAN 차이점

ELECTRA는 **GAN**과 유사하지만 동일하지 않음

GAN

- Generator & Discriminator **적대적 학습**
- 완전한 노이즈 벡터 입력
- 학습 불안정

ELECTRA

- Generator는 **MLM 방식**
- Discriminator만이 핵심 학습 대상
- Generator가 **정답 토큰을 맞히면 "원본"으로 취급**
- **적대적 학습**

➡ 안정적 + 효율적

6. Generator 크기가 작은 이유

- Generator가 너무 강하면
→ Discriminator가 학습할 정보가 줄어들음
- 그래서 ELECTRA는:

- Generator 크기 = Discriminator의 $1/2 \sim 1/4$
- 임베딩만 공유, 전체 가중치는 공유

📌 목적:

| “적당히 틀리는 Generator”

7. 사전 학습 후 실제 사용 모델

- Generator 사용 안 함
- Discriminator만 사용

➡ 구조적으로 BERT와 완전히 동일한 Encoder 모델

8. 다운스트림 작업 방식

- BERT와 동일
 - 문장 분류
 - 감성 분석
 - 자연어 추론
- 동일한 Fine-tuning 방식 적용

📌 차이점은 사전 학습 방식뿐

9. KoELECTRA (한국어 ELECTRA)

✓ 제공 모델

- `monologg/koelectra-small-v3` (14M)
- `monologg/koelectra-base-v3` (110M)

✓ 실습에서 사용한 모델

`monologg/koelectra-base-v3-discriminator`

- BERT-base와 동일한 파라미터 수
- 성능 비교 목적에 적합

10. KoELECTRA 모델 구조

✓ Embedding

- word embedding
- position embedding
- token type embedding
- LayerNorm + Dropout

✓ Encoder

- Transformer Encoder **12층**
- BERT와 동일 구조

✓ Classifier

- [CLS] 토큰 벡터 사용
 - Dense → Dropout → Output projection
-

11. 실험 결과 해석

✓ 성능 결과

모델	Test Accuracy
BERT	80.85%
KoELECTRA	87.32%

📌 약 +7.5% 성능 향상

사전학습 효율성의 힘

12. GLUE 벤치마크 결과

- ELECTRA는 ****모든 크기(small / base / large)****에서
→ BERT보다 높은 점수
- 동일한 파라미터 기준:
 - small: +5%

- base: +3%
- large: +2%

📌 특히 학습 시간 대비 성능이 압도적

13. ELECTRA의 장점 요약

모든 토큰에서 학습 신호

빠른 수렴

적은 데이터·짧은 시간에도 높은 성능

BERT 구조 그대로 사용 가능

14. ELECTRA의 한계

생성 작업에는 부적합

Generator는 학습용 보조 역할

요약·번역 같은 생성 작업은 BART/GPT 계열이 적합

◆ T5 (Text-to-Text Transfer Transformer) 개념 정리

1. T5란?

T5 (Text-to-Text Transfer Transformer)

- 2019년 **Google**이 발표한 Transformer 기반 NLP 모델
- 핵심 아이디어:

모든 NLP 작업을 “텍스트 → 텍스트” 문제로 통일

📌 입력도 텍스트, 출력도 텍스트

📌 분류·번역·요약·QA 전부 같은 형태로 처리

2. T5의 가장 큰 특징: Text-to-Text 패러다임

기존 NLP 모델

- 입력 → 벡터/확률
- 출력 → 클래스, 점수, 일부 토큰

T5

- 입력: 텍스트
- 출력: 텍스트

 예시

작업	입력	출력
번역	translate English to Korean: This is good.	그것은 좋다.
문법 판단	cola: 수업이 잘 점프하고 있다.	적합하지 않음
유사도	sts: 문장1 문장2	3.8
요약	summarize: 뉴스 본문	요약 문장

 모든 작업을 하나의 모델로 처리 가능

3. T5 모델 구조

✓ Encoder-Decoder 구조

- **Encoder:** 입력 문장 이해
- **Decoder:** 출력 문장 생성

 구조 자체는 **BART**와 유사

하지만 **문제 정의 방식(Text-to-Text)**이 핵심 차이

4. T5가 잘하는 작업

- 번역
- 문서 요약
- 질의응답(QA)
- 문장 유사도
- 문장 분류

“출력이 문장인 작업”에 최적

5. T5 사전 학습 데이터

- **C4 (Colossal Clean Crawled Corpus)**
 - 웹 크롤링 대규모 말뭉치
 - 정제된 영어 텍스트

다양한 자연어 태스크에 일반화 가능

6. T5 사전 학습 방식 (핵심)

Span Corruption (구간 마스킹)

- 문장의 연속된 토큰 구간(span) 을 마스킹
- BERT처럼 단어 하나가 아니라 구문 단위

7. 센티넬 토큰 (Sentinel Token)

개념

- `<extra_id_0>` , `<extra_id_1>` , ... `<extra_id_99>`
- 각 마스킹 구간을 대표하는 토큰

예시

원문:

인코더-디코더 모델 구조

마스킹:

`<extra_id_0>` 모델 `<extra_id_1>`

출력:

인코더 `<extra_id_0>` 디코더 `<extra_id_1>`

특징

- 마스크 위치 + 복원 대상 동시에 학습

- 디코더는 **마스크된 토큰을 순서대로 생성**

8. T5 사전 학습 목표

센티널 토큰을 기준으로 마스크된 텍스트 구간을 정확히 생성

➡ 자연어 생성 능력 매우 강함

9. Fine-tuning (미세 조정) 방식

✓ 지도 학습

- 입력: 작업 설명 + 텍스트
- 출력: 정답 텍스트

작업 토큰 예시

```
summarize:  
translate English to German:
```

모델이 작업 자체를 학습

10. 실습: 뉴스 요약 데이터 구성

✓ 데이터셋

- `argilla/news-summary`
- `text` → 뉴스 본문
- `prediction` → 요약문

✓ 입력 전처리

```
df["text"] = "summarize: " + df["text"]
```

이유

“이 입력은 요약 작업이다” 라고 명시

11. T5 토크나이징 특징

- `T5Tokenizer`
- padding + truncation 사용
- `max_length=128`

✓ 토큰 예시

- `21603`, `10` → `"summarize:"`
- 출력에서 확인 가능

12. T5 학습 입력 구성

모델 입력

- `input_ids`
- `attention_mask`
- `decoder_input_ids`
- `labels`

decoder_input_ids & labels 관계

- `decoder_input_ids`
→ target에서 **마지막 토큰 제거**
- `labels`
→ target에서 **첫 토큰 제거**
- pad 토큰은 `100` 으로 설정 → loss 계산 제외

Teacher Forcing 구조

13. T5 모델 선언

```
T5ForConditionalGeneration.from_pretrained("t5-small")
```

- `t5-small` : 가장 작은 버전
- Encoder-Decoder 구조 유지

- shared embedding 사용

14. T5 모델 구조 특징

```
shared embedding
encoder (6 layers)
decoder (6 layers)
lm_head
```

✓ shared embedding

- Encoder / Decoder 임베딩 공유
- 파라미터 수 감소
- 일반화 성능 향상

15. 학습 결과 해석

✓ 손실 변화

```
Epoch 1: Train Loss 4.31 → Val 3.31
Epoch 3: Train Loss 3.12 → Val 2.76
```

➡ 손실 지속 감소

➡ 정상적인 학습 진행

16. 생성 테스트 (Inference)

```
model.generate(
    num_beams=3,
    repetition_penalty=2.5,
    length_penalty=1.0
)
```

주요 파라미터

- `num_beams`: Beam Search

- `repetition_penalty` : 반복 방지
- `length_penalty` : 길이 조절

➡ 요약 품질 향상

17. T5의 장점 요약

모든 NLP 작업을 하나의 형식으로 처리

Encoder-Decoder 기반 생성 능력 우수

작업 토큰으로 확장성 높음

요약·번역·QA에 매우 강함

18. T5의 한계

분류 전용 모델(BERT, ELECTRA)보다 느림

출력이 텍스트 → 후처리 필요

계산량 큼
